Lab 06: Çarpışma Algılama ve Fizik Temelleri

PyGame ile Oyun Geliştirme – Hafta 6

Ad Soyad: _____	**Numara:** _____	**Tarih:** _____

1 Çalışma 1: Rect Çarpışması ve collidepoint()

Örnek Kod

Aşağıdaki programı `rect_carpisma.py` dosyasına yazın ve çalıştırın:

```python
import pygame

pygame.init()
GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Rect Carpisma Testi")
saat = pygame.time.Clock()
font = pygame.font.SysFont("Arial", 20)

# Iki dikdortgen
oyuncu = pygame.Rect(100, 250, 60, 60)
engel = pygame.Rect(350, 250, 80, 80)
hiz = 4

calistir = True
while calistir:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            calistir = False

    # Hareket
    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_LEFT]:
        oyuncu.x -= hiz
    if tuslar[pygame.K_RIGHT]:
        oyuncu.x += hiz
    if tuslar[pygame.K_UP]:
        oyuncu.y -= hiz
    if tuslar[pygame.K_DOWN]:
        oyuncu.y += hiz

    # Carpisma kontrolu
    carpisma = oyuncu.colliderect(engel)

    # Fare nokta kontrolu
    fare_x, fare_y = pygame.mouse.get_pos()
    fare_engelde = engel.collidepoint(fare_x, fare_y)

    # Cizim
    ekran.fill((20, 20, 40))

    renk_engel = (231, 76, 60) if carpisma else (46, 204, 113)
    renk_oyuncu = (231, 76, 60) if carpisma else (52, 152, 219)

    pygame.draw.rect(ekran, renk_engel, engel)
    pygame.draw.rect(ekran, renk_oyuncu, oyuncu)

    # Fare engelin uzerinde mi?
    if fare_engelde:
        pygame.draw.rect(ekran, (241, 196, 15), engel, 3)
```

colliderect() ve collidepoint()

`colliderect()` iki dikdörtgenin örtüşüp örtüşmediğini kontrol eder. `collidepoint(x, y)` ise bir noktanın dikdörtgenin içinde olup olmadığını kontrol eder. Her ikisi de `True` veya `False` döndürür.

Talep 1: Üç Engel

Tek engel yerine 3 farklı konumda ve boyutta engel oluşturun. Her birini bir listeye ekleyin ve `for` döngüsüyle tüm engellere karşı çarpışma kontrolü yapın. Çarpışılan engelin rengi kırmızıya dönsün.

Talep 2: Çarpışma Sayacı

Her yeni çarpışma olduğunda bir sayaç artırsın (aynı engele tekrar tekrar değmek sayılmamalı). Sağ üst köşede "Carpisma: 5" şeklinde toplam çarpışma sayısını gösterin. `onceki_carpisma` adlı bir değişkenle önceki karenin durumunu takip edebilirsiniz.

Talep 3: Fare Tıklama ile Engel Ekleme

Fare sol tuşuyla tıklanan konuma yeni bir engel ekleyin (`MOUSEBUTTONDOWN` olayı). Her yeni engelin boyutu 40 ile 100 piksel arasında rastgele olsun. Sağ tıklamayla fare pozisyonundaki engeli silin (`collidepoint()` ile hangi engele tıklandığını bulun).

2 Çalışma 2: Sprite Çarpışma Fonksiyonları

Örnek Kod

Aşağıdaki programı `sprite_carpisma.py` dosyasına yazın ve çalıştırın:

```
import pygame
import random

pygame.init()
GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Sprite Carpisma")
saat = pygame.time.Clock()
font = pygame.font.SysFont("Arial", 20)

class Oyuncu(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((50, 50), pygame.SRCALPHA)
        pygame.draw.rect(self.image, (52, 152, 219),
                         (0, 0, 50, 50), border_radius=5)
        self.rect = self.image.get_rect(
            center=(GENISLIK // 2, YUKSEKLIK - 80))

    def update(self):
        tuslar = pygame.key.get_pressed()
        if tuslar[pygame.K_LEFT]:
            self.rect.x -= 5
        if tuslar[pygame.K_RIGHT]:
            self.rect.x += 5
        if tuslar[pygame.K_UP]:
            self.rect.y -= 5
        if tuslar[pygame.K_DOWN]:
            self.rect.y += 5
        self.rect.clamp_ip(ekran.get_rect())

class Dusman(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((30, 30), pygame.SRCALPHA)
        pygame.draw.rect(self.image, (231, 76, 60),
                         (0, 0, 30, 30), border_radius=3)
        self.rect = self.image.get_rect(
            center=(random.randint(50, 750),
                    random.randint(-200, -30)))
        self.hiz = random.uniform(1.5, 4.0)

    def update(self):
        self.rect.y += self.hiz
        if self.rect.top > YUKSEKLIK:
            self.kill()

class Mermi(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((4, 12), pygame.SRCALPHA)
```

spritecollide ve groupcollide

`spritecollide(sprite, group, dokill)` tek bir Sprite ile bir grubu karşılaştırır. `groupcollide(group1, group2, dokill1, dokill2)` ise iki grubu birbirine karşılaştırır. `dokill` parametresi `True` ise çarpışan Sprite otomatik silinir.

Talep 1: Can Sistemi Ekleyin

Oyuncuya 3 can verin. Düşmana çarpınca 1 can kaybedin (skor düşürmek yerine). Canlar bitince "GAME OVER" yazısı gösterin ve oyunu durdurun. Ekranın sağ üst köşesine kalan canları gösterin.

Talep 2: Mermi Soğuma Süresi (Cooldown)

Space tuşuna basılı tutunca sürekli mermi atılmasını engelleyin. `pygame.time.get_ticks()` kullanarak 250 ms'lik bir bekleme süresi ekleyin. Son mermi zamanını `son_mermi` değişkeninde saklayın.

```
simdiki_zaman = pygame.time.get_ticks()
if simdiki_zaman - son_mermi > 250:
    # Mermi olustur
    son_mermi = simdiki_zaman
```

Talep 3: Farklı Düşman Türleri

İki farklı düşman türü oluşturun: yavaş ama büyük (50x50, hız 1-2) ve hızlı ama küçük (20x20, hız 3-5). Her düşman oluştururken rastgele seçim yapın. Büyük düşmanı vurmak 3 puan, küçük düşmanı vurmak 1 puan versin. Düşman türüne göre renk farklı olsun.

3 Çalışma 3: Yerçekimi ve Fizik Simülasyonu

Örnek Kod

Aşağıdaki programı `yercekimi_demo.py` dosyasına yazın ve çalıştırın:

```python
import pygame

pygame.init()
GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Yercekimi Simulasyonu")
saat = pygame.time.Clock()
font = pygame.font.SysFont("Arial", 18)

# Fizik sabitleri
YERCEKIMI = 0.5
SURTUNME = 0.98
ZEMIN_Y = 550

class Top(pygame.sprite.Sprite):
    def __init__(self, x, y, renk=(52, 152, 219)):
        super().__init__()
        self.yaricap = 15
        self.image = pygame.Surface(
            (self.yaricap * 2, self.yaricap * 2), pygame.SRCALPHA)
        pygame.draw.circle(self.image, renk,
                           (self.yaricap, self.yaricap), self.yaricap)
        self.rect = self.image.get_rect(center=(x, y))
        self.hiz_x = 0.0
        self.hiz_y = 0.0

    def update(self):
        # Yercekimi uygula
        self.hiz_y += YERCEKIMI

        # Surtunme (yatay)
        self.hiz_x *= SURTUNME

        # Konumu guncelle
        self.rect.x += self.hiz_x
        self.rect.y += self.hiz_y

        # Zemin carpisma ve geri sekme
        if self.rect.bottom >= ZEMIN_Y:
            self.rect.bottom = ZEMIN_Y
            self.hiz_y *= -0.7  # Enerji kaybi ile sekme

        # Duvar carpisma
        if self.rect.left < 0:
            self.rect.left = 0
            self.hiz_x *= -0.8
        elif self.rect.right > GENISLIK:
            self.rect.right = GENISLIK
            self.hiz_x *= -0.8

# Gruplar
```

Enerji Kaybı

Geri sekmede `hiz_y *= -0.7` çarpanı topun her sekmede enerjisinin %30'unu kaybetmesini sağlar. Bu değer 1.0 olursa sonsuz sekme, 0.0 olursa hiç sekmeme olur. Oyununuza göre ayarlayın.

Talep 1: Yerçekimi Değiştirme

Yukarı/aşağı ok tuşlarıyla `YERCEKIMI` değerini 0.1 ile 2.0 arasında artırıp azaltın. Mevcut değeri ekranda gösterin. Ay yerçekimi (0.08) ve Jüpiter yerçekimi (1.3) ile deneyip farklılıkları gözlemleyin.

Talep 2: Sürtünme Etkisi

Zemine temas eden toplar için ek sürtünme uygulayın: zemine değdiğinde `hiz_x *= 0.92` ile yatay hızı azaltın. Hız çok küçüldüğünde (`abs(hiz_x) < 0.1` ve `abs(hiz_y) < 0.5`) topu tamamen durdurun.

Talep 3: Toplar Arası Çarpışma

`spritecollide()` kullanarak topların birbirine çarpmasını algılayın. Çarpışan iki top birbirinden uzaklaşsın: hız vektörlerini ters çevirin.

```
for top in tum_spritelar:
    carpisan = pygame.sprite.spritecollide(
        top, tum_spritelar, False)
    for diger in carpisan:
        if diger != top:
            # Hiz degistir
            top.hiz_x, diger.hiz_x = diger.hiz_x, top.hiz_x
            top.hiz_y, diger.hiz_y = diger.hiz_y, top.hiz_y
```

4 Çalışma 4: Mini Proje – Breakout (Tuğla Kırma)

Proje Gereksinimleri

Kitaptaki Breakout (Tuğla Kırma) projesini aşağıdaki gereksinimlere göre oluşturun:

- Ok tuşları ile kontrol edilen **Raket** (120x15 piksel, ekranın altında)
- Duvarlardan ve raketten seken bir **Top** (yarıçap 8 piksel)
- En az 5 sütun $\times$ 4 satır = 20 adet **Tuğla**
- `spritecollide()` ile top-tuğla çarpışması
- Top ekranın altından düşerse can kaybı
- Skor göstergesi (her tuğla = 10 puan)

Top Sekme Mantığı

Top duvara veya rakete çarptığında yön değiştirir:

```
# Yan duvarlara carpma
if self.rect.left <= 0 or self.rect.right >= GENISLIK:
    self.hiz_x *= -1

# Tavana carpma
if self.rect.top <= 0:
    self.hiz_y *= -1

# Rakete carpma
if self.rect.colliderect(raket.rect):
    self.hiz_y *= -1
    self.rect.bottom = raket.rect.top
```

Talep 1: Temel Yapıyı Oluşturun

Raket, Top ve Tuğla sınıflarını yazın. Raket ok tuşlarıyla hareket etsin, top sabit hızla hareket edip duvarlardan seksin, tuğlalar renkli satırlar halinde dizilsin. Henüz çarpışma kontrolü yapmayın, sadece görsel olarak sahneyi oluşturun.

Talep 2: Çarpışma ve Skor

Top-tuğla çarpışmasını `spritecollide()` ile ekleyin. Çarpışan tuğla silinsin ve skor 10 artsın. Top-raket çarpışmasını `colliderect()` ile yapın. Topun rakete çarpma noktasına göre açısını değiştirin:

```
# Raketin neresine vurdu?
vurun_x = top.rect.centerx - raket.rect.left
oran = vurun_x / raket.rect.width  # 0.0 - 1.0
# Aciyi hesapla (-60 ile +60 derece arasi)
import math
aci = (oran - 0.5) * 120
top.hiz_x = math.sin(math.radians(aci)) * hiz
top.hiz_y = -abs(math.cos(math.radians(aci)) * hiz)
```

Talep 3: Can ve Oyun Durumu

Oyuncuya 3 can verin. Top ekranın altından düşerse 1 can azalsın ve top raketin üstünde yeniden başlasın (Space ile fırlatma). Canlar bitince “GAME OVER” ekranı gösterin. Tüm tuğlalar kırılınca “KAZANDINIZ!” mesajı gösterin.

Bonus: Güçlendirme (Power-up) Sistemi

Bazı tuğlalar kırıldığında aşağı doğru düşen güçlendirmeler bıraksın. Raket ile yakalanınca etki uygulansın:

- **Geniş Raket:** Raket genişliği 2 katına çıksın (5 saniye süreyle)
- **Çoklu Top:** Mevcut top 3'e çoğalsın
- **Ekstra Can:** Oyuncu 1 can kazansın

Güçlendirme Sprite'ı yavaşça düşsün ve zemine ulaşırsa `kill()` ile silinsin. Farklı güçlendirmeleri farklı renkle gösterin.

Başarılar!